International Journal of Software Engineering
and Knowledge Engineering
Vol. 25, No. 5 (2015) 887–907
© World Scientific Publishing Company
DOI: 10.1142/S0218194015400252

SyncAS: A Virtual Block Approach to Tame Asynchronous Programming

Hiroaki Fukuda

Information Science and Engineering
Shibaura Institute of Technology
3-7-5 Koto, Toyosu, Tokyo 135-8548, Japan
hiroaki@shibaura-it.ac.jp

Paul Leger

Escuela de Ciencias Empresariales
Universidad Católica del Norte, Chile
pleger@ucn.cl

Asynchronous programming has been widely adopted in domains such as Web development. This programming style usually uses callback methods, non-blocking operations, allowing high responsive user interactions even if an application works without multi-threading. However, this style requires the uncoupling of a module into two sub-modules at least, which are not intuitively connected by a callback method. The separation of modules spurs the birth of other issues: *callback spaghetti* and *callback hell*. This paper proposes a *virtual block* approach to address the previous two issues. This approach enables a programmer to virtually block a program execution and restart it at arbitrary points in the program. As a result, programmers do not need to uncouple a module even if non-blocking operations are adopted; therefore, callback dependencies disappear. Using aspect-oriented programming, this approach uses aspects to control the execution of a program in an oblivious manner. As a consequence, programmers do not need to be concerned whether pieces of code use blocking or non-blocking operations. We implement a proof-of-concept for this approach, called SyncAS, for ActionScript3. In addition, we apply our proposal to a toy application and conduct experiments to show its modular application, flexibility, and performance.

Keywords: SyncAS; virtual block; asynchronous programming; aspect-oriented programming.

1. Introduction

Modularity is crucial to implement large-scale software because it allows developers to decompose a software into several modules, enhancing maintainability, adaptability, and reusability of software [19]. For example, in object-oriented programming, a class (i.e. module) gives interfaces that allow other classes to use it while hiding implementation details.

Asynchronous programming has been adopted for a variety of reasons: hiding latency of the network improves the responsiveness in distributed applications, avoiding resource costs of creating many threads increases throughput of an application. Developers of interactive applications, like those found on the Web, widely use this programming style. The basic idea of asynchronous programming is to decompose a blocking operation that waits for its completion into a non-blocking operation that immediately returns control. Whenever the non-blocking operation execution ends, a piece of code known as *callback method*, is invoked. The use of a callback method requires dividing a sequential set of operations into executions of methods that must wait the completion of non-blocking operations.

The use of callbacks in asynchronous programming comes with issues that affect the modular development of software. The most notable issues are *callback spaghetti* [16] and *callback hell* [17]. Callback spaghetti refers to the concern implementation that has a complex and tangled control structure because of continuations over callback methods. Callback hell refers to deeply-nested callbacks that have dependencies on data returned from previous asynchronous invocations.

This paper proposes a novel approach called *virtual block* to modularly use asynchronous programming,[a] and thus addressing issues of callback spaghetti and hell. Similar to synchronous programming, our approach blocks the current method execution, but the distinguishing feature is that the rest of program is still working (e.g. event handlers, timers, animations). We implement this approach in SyncAS, a prototype library for ActionScript3: an object-oriented language with first-class functions. With SyncAS, a programmer can use a module that utilizes non-blocking operations in a synchronous fashion, meaning that the programmer does not need to be concerned whether synchronous or asynchronous executions. Using Aspect-Oriented Programming (AOP) [12], a programmer modularly specifies when a program execution should virtually be blocked and/or restarted. As the SyncAS implementation follows the ECMAScript standard, our approach should work on any ECMAScript language.

Paper roadmap. Section 2 gives a brief introduction to asynchronous programming and its problems. Section 3 conceptually explains SyncAS through an example, and Sec. 4 details its implementation. Section 5 evaluates and compares this proposal. Sec. 6 discusses related work and Sec. 7 presents conclusions and future work.

Availability. SyncAS is available online at http://www.pragmatic.jp/research/syncas. SyncAS currently supports the Firefox, Safari, Chrome, and Opera browsers without the need of an extension.

[a] Two earlier versions of the main matter of this article appeared in the *Proceedings of Modularization for Practical Software Engineering (MPSE 2014)* as a short paper [1] and *29th Annual ACM Symposium on Applied Computing (SAC 2015)* as a poster [8].

2. Asynchronous Programming Problems

Asynchronous programming is now widely-adopted between mainstream programmers [3]. This section briefly describes asynchronous programming and its two main difficulties: *Callback Spaghetti* [16] and *Callback Hell* [17].

2.1. *Asynchronous programming*

The basic principle behind asynchronous programming is to decompose a synchronous operation that blocks the software execution until its completion, into a non-blocking operation and a *callback method*. The non-blocking operation immediately returns, and the callback method is invoked when this operation is finished [3]. Callback methods are widely used for asynchronous programming. Listing 1 shows an example of the use of a callback method in ActionScript3. The method showImageFromURL includes a non-blocking operation (Line 4), meaning that a certain callback method must to be associated to get the result of download (Line 3). To carry out this association, a method called addEventListener is used. The first argument of this method is an event type (Downloader.Complete) and the second is the callback method reference (callback). In this way, a synchronous operation is decomposed into two methods (lines 3 and 4). If the asynchronous operation controls a state of the application, asynchronous programming may require global flags. For example in this listing, a flag variable is required to indicate if the image was downloaded correctly (more on this in the following section).

2.2. *Callback spaghetti*

The difficulty with using callback methods is in converting a particular synchronous call-site into an asynchronous call-site. This is because the transformation requires a programmer to represent the continuation of the original site as a callback method. We illustrate the issues associated with callback methods using an extension of the previous example, which checks the image integrity before showing it.

```
1  function showImageFromURL(url:URL):void {
2    var dl = new Downloader();
3    dl.addEventListener(Downloader.Complete, callback);
4    dl.download(url);
5  }
6
7  function callback(e:Event) {
8    var img:Image = convertToImage(e.data);
9    show(img);
10 }
```

Listing 1. Asynchronous programming in ActionScript.

```
class ImageViewer {

  function showFromURL(url:URL,checksum:Function):void {
    var e:Event = new Request().send(url);
    var img:Image = checkAndConvertToImage(e,checksum);
    show(img);
  }

  function checkAndConvertToImage(e:Event,checksum:Function) {
    if (checksum(e.data))
       return convertToImage(e.data);
    else //throw an exception
} }


class Request {

   function send(url:URL):Event {
     return new Downloader().download(url);
 } }
```

Listing 2. A synchronous version for a remote image viewer.

Listing 2 shows two classes: ImageViewer and Request. The ImageViewer class contains two methods: showFromURL and checkAndConvertToImage. The first method downloads data from a url and uses the second method to convert this data to an image if it passes an integrity check. The Request class has the send method that actually downloads some data by using the download method of the Downloader class. For this example, we assume download is a blocking operation, which takes a significant period of time.

Because of several advantages such as reusability, maintainability and adaptability, dividing a system into a composition of modules is natural [19]. Therefore, in Listing 2, Request encapsulates how to get data from sources. Now, suppose that the send method changes from blocking to non-blocking to carry out the data downloaded: downloadAsync. Listing 3 shows the rewritten program of Listing 2 for this change. Three major changes are found in Listing 3. First, invoking checkAndConvertToImage is removed from showFromURL because send, that invokes downloadAsync, returns immediately without any data. Instead, the reference of the showFromURL method is passed as a callback by using next, which is defined in Request. Second, a variable checksum must be defined in ImageViewer to keep the argument checksum in showFromURL because checkAndConvertToImage only accepts one parameter. Third, the show function call must be moved to checkAndConvertToImage because showFromURL does not contain the image. As described previously in this section, a module that uses an asynchronous method requires representing the continuation as a callback. As a consequence, if the location of the

```
class ImageViewer {
  var checksum:Function;

  function showFromURL(url:URL, checksum:Function):void {
    this.checksum = checksum;
    var request = new Request().next(checkAndConvertToImage);
    request.send(url);
  }

  function checkAndconvertToImage(e:Event):Image {
    if (this.checksum(e.data)) {
      var img:Image = convertToImage(e.data);
      show(img);
    }
    else //throw an exception
} }

class Request {
  var nextF:Function;
  function next(f:Function):void {
    nextF = f;
  }

  function send(url:URL):void {
    var dl = new Downloader();
    dl.addEventListener(Downloader.Complete, callback);
    dl.downloadAsync(url);
  }

  function callback(e:Event):void {
      nextF(e);
} }
```

Listing 3. An asynchronous version for a remote image viewer.

continuation is far from the call-site, understanding control flow is difficult, leading to callback spaghetti. Moreover, a change of implementation inside of one module (e.g. Request) may require call-site modifications in other modules (e.g. ImageViewer), breaking modular principles.

2.3. *Callback hell*

The use of nested anonymous functions is a solution to prevent callback spaghetti when non-blocking operations are adopted in asynchronous programming. Listing 4 uses an anonymous function instead of checkAndconvertToImage in Listing 3.

A programmer can write a callback method directly inside a call-site, making a control flow easier to understand. However, the definition and immediate execution of an anonymous function does not allow developers to reuse it. Moreover, nested callback methods in large-scale applications leads to a mix of collocated code fragments and creates complex control flows [10].

```
class ImageViewer {

  function showFromURL (url:URL,checksum:Function):void {
    var request = new Request().next(
      function(e:Event) {
        if (checksum(e.data)) {
          var img:Image = convertToImage(e.data);
          show(img);
        }
        else //throw an exception
      });

    request.send(url);
} }
```

Listing 4. A synchronous version with nested callbacks for the remote image viewer.

3. SyncAs

This section presents the description of our approach, reviewing its basic idea: the separation of base concerns of a program and its control flow management. From this section, we define *async method* as a method that at least contains a non-blocking operation, and *sync method* as a method that contains only blocking operations. For example, in Listing 3, the send method is an async method because downloadAsync is a non-blocking operation. The next section introduces SyncAs by an example. Remaining sections explain how SyncAs works.

3.1. *Using SyncAS to virtually block a method execution*

As mentioned in Sec. 2, modular programming is adequate for developing large-scale applications because of several advantages (e.g. reusability). In addition, synchronous programming style makes it easy to understand control flows. However, introducing asynchronous programming requires several changes such as dividing one method into a call-site and its continuations: a reference to the next expression of a program execution. Then passing a continuation to another module as a callback makes complex control flows. Therefore, our *virtual block* approach enables a programmer to write a program in a synchronous style while the program is executed asynchronously.

Figure 1 shows how an application with virtual block behaves. In this figure, *sync* and *async* represent invocations of sync and async methods respectively. With our virtual block approach, after an async invocation, the rest of invocations are not executed but kept as continuations. These continuations are executed when the async invocation ends.

We now rewrite Listing 3 with our virtual block approach in Listing 5. The ImageViewer class now has the same implementation of the synchronized version of the image viewer (Listing 2). The program execution is virtually blocked after Line 3, and the remaining executions are stored as continuations (in this piece of code, Line 4). These continuations are executed after callback is executed in Line 16.

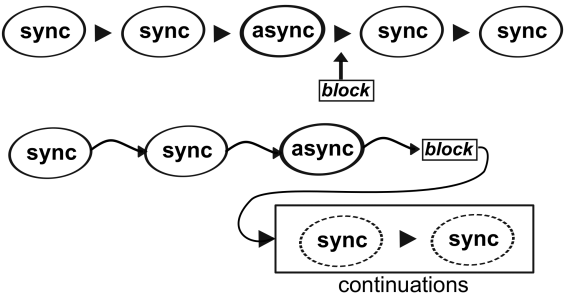

Fig. 1. Virtual block approach.

```
1  class ImageViewer {
2    function showFromURL(url:URL,checksum:Function):void {
3      var e:Event = new Request().send(url); //SyncAS virtually blocks the execution
4      var img:Image = checkAndConvertToImage(e.data,checksum);
5      show(img);
6    }
7  }
8
9  class Request {
10   function send(url:URL):Event {
11     var dl = new Downloader();
12     dl.addEventListener(Downloader.Complete,callback);
13     dl.downloadAsync(url);
14   }
15
16   function callback(e:Event) {
17     //SyncAS restarts the execution
18   }
19 }
```

Listing 5. A SyncAS version for the remote image viewer.

In Listing 5, each method looks synchronous, however send behaves asynchronously. Note that a block in our approach does not mean a real blocking operation, meaning that the ActionScript runtime engine does not block any operation. As a result, a user can continue using an application without waiting for non-blocking operation executions. To deploy the SyncAS mechanism necessary for Listing 5, programmers only need to code the line below, where first parameter represents the point in which the execution should be virtually blocked and the second one refers to when the execution restarts.

```
SyncAS.addAsyncOperation("Request.send","Request.callback");
```

3.2. *Blocking and restarting an execution*

SyncAS requires a programmer to specify when the execution of a program is blocked and restarted. In other words, the programmer needs to specify two executions points

of a program: (a) when it is blocked and (b) when it is restarted. The problem is how to specify these points. We now have two design choices: explicit-specification and implicit-specification. Suppose two programmers P1 and P2 to implement a *remote image viewer*. P1 and P2 independently implement ImageViewer and Request classes of Listing 2 respectively. Based on the discussions in Sec. 2.2, P1 and P2 cannot work independently even if they never change the interface of each class (e.g. method names) because of replacing a sync method with an async method (e.g. download replaces with downloadAsync). Although SyncAS adopts implicit-specification, we next discuss both kinds of specifications for asynchronous programming.

Explicit-specification. Explicit-specification means that a programmer explicitly writes a block that resembles *labeled* in the call-site. For example, in C# [2], developers must write the construct async before an async method definition. With this explicit specification, a runtime system can know when it has to keep statements in a queue as continuations. Although this approach enables a runtime system to know when it will change its behavior such as evaluating or keeping continuations, implementing applications may be complicated. This is because a programmer (e.g. P1) who uses a method (e.g. send) needs to know the details of its implementation, meaning that a change done by another programmer (e.g. P2) affects all call-sites.

Implicit-specification. Conversely, implicit-specification means that a programmer, who uses an async method, does not explicitly specify when an async execution starts. With this approach, P1 does not have to rewrite a block even if the send implementation has changed. However, P2 must write two pieces of code to indicate when a program execution is blocked and restarted. The block of a program execution must be written in the ImageViewer class while the corresponding restart should be done in Request. As a consequence, a block and its corresponding restart must be used as a pair, resulting in a crosscutting concern. To modularize this crosscutting concern, SyncAS adopts Aspect-Oriented Programming (AOP) [12]. A block and its corresponding restart need to be used as a pair, meaning that they should be a module. With the creation of this kind of module, the programmer P2 can write this code without the need of P1 modifying its implementations, enhancing maintainability of a software.

3.3. *An aspect to control flow*

The key technique behind the SyncAS API is AOP. AOP allows programmers to use aspects, as embodied in e.g. AspectJ [11], to modularize crosscutting concerns. In the pointcut-advice model of AOP [15], an aspect specifies program execution points of interest, named (dynamic) *join points*, through predicates called *pointcuts*. When an aspect matches a join point, it takes an action, called *advice*.

Listing 6 shows a pseudo implementation of the SyncAS API applied to Listing 5. In SyncAS, an aspect is composed of a list of 3-tuples: a pointcut, an advice, and an advice kind. For example, in Listing 6, pcBlock/pcRestart and advBlock/advRestart

```
1  // pointcut and advice to block an execution
2  var pcBlock = function(jp, env) {
3    return jp.className == "Request" && jp.methodName == "send";
4  }
5
6  var advBlock = function(jp, env) {
7    SyncAS.blockJP(jp);
8    env.add("blockJp", jp);
9  }
10
11 // pointcut and advice to restart an execution
12 var pcRestart = function(jp, env):* {
13   return jp.className == "Request" &&
14          jp.methodName == "callback";
15 }
16
17 var advRestart = function(jp, env) {
18   SyncAS.restartJP(env.get("blockJp"));
19 }
20
21 var virtualBlock = new Aspect([pcBlock, advBlock, AFTER],
22                              [pcRestart, advRestart, AFTER]);
23 SyncAS.deploy(virtualBlock);
```

Listing 6. The aspect used by SyncAS to control a program execution.

correspond to pointcuts and advices respectively. The jp object refers to a join point. To expose context information, a pointcut and an advice can take an extra parameter as environment (env) that is shared inside of an aspect. For example, the virtualBlock aspect stores the blocked join points. Finally, SyncAS provides a deploy method to activate an aspect.

We now review the Listing 6 in detail. The piece of code expresses two pairs of pointcuts and advices: one is for blocking (lines 2–10) and another for restarting (lines 12–20). For the first pointcut-advice pair, the advice is invoked after the execution Request.send. In this advice, SyncAS.blockJP blocks the execution of the matched join point, which is stored in an environment of bindings. After this advice is executed, the SyncAS runtime stores the next and unexecuted join points of the method where send was called. As a consequence, SyncAS keeps statements of lines 4 and 5 as continuations of Listing 5. For the second pointcut-advice pair, the advRestart advice is invoked after the execution of Request.complete. As a result, the blocked join points are restarted by using SyncAS.restartJP. In Line 21, two pairs of pointcuts and advices consist of an aspect that is deployed in Line 23. In this way, an aspect enables a programmer to encapsulate pieces of code that control a flow in one place.

4. Implementation

This section details the prototype implementation of SyncAS. The implementation of SyncAS borrows ideas from AspectScript [20], an aspect language for JavaScript. Thereby, SyncAS mainly consists of three phases: code transformation, weaving

process, and runtime. Sec. 4.1 details the code transformation, and Sec. 4.2 explains the weaving process. Finally, Sec. 4.3 elaborates SyncAS runtime.

The current version of SyncAS is implemented in/for ActionScript, which provides a number of non-blocking operations. Because this language possesses dynamic features (e.g. first-class functions), the SyncAS weaving is mostly done at runtime. In order to weave aspects dynamically without modifying the ActionScript runtime, the SyncAS implementation first performs a code transformation in which every statement is rewritten into invocations of functions. These invocations make it possible to perform runtime aspect weaving and manage control flows by a set of blocks and restarts. Furthermore, the key feature that enables the SyncAS approach is *closure*: an anonymous function with access to their declaring lexical scope. As the SyncAS implementation follows the ECMAScript standard, our approach should work on any ECMAScript language.

4.1. *Code transformation*

In the current implementation, the code transformation is only applied to a subset of ActionScript syntax like statements, object creations, method invocations, and method definitions. This transformation allows us to have a join point model that supports join points for aforementioned statements and expressions. This kind of join points is enough to realize the proof-of-concept of SyncAS. This transformation wraps an original piece of code in a closure. Although this transformation should be performed automatically, the current implementation does not support it yet. Tools like MetaAS [9] can be used to implement the transformation.

The transformation rules are defined in Fig. 2. The left and right side in Fig. 2 represent a subset of ActionScrip3 and transformation rules respectively. The words **call-s**, **call**, and **wrap** represent predefined functions provided by SyncAS to create join points. In addition, **arguments** (in short **args** in Fig. 2) is a reserved keyword in ActionScript that represents arguments in the callee method. Finally, *cn* and *mn* represent a class name and a method name respectively that are used to specify join points.

Rule 1 rewrites a statement (e.g. if, while) to a function call by using call-s, meaning that a statement can be a join point while keeping its environment. A variable declaration with an assignment shown in rule 2 is divided into two parts: a variable definition and an assignment wrapped by a function. The object creation that uses the new keyword is used with an assignment. Therefore, Rule 3 is a special case of rule 2 in which the expression is an object creation. Rule 4 and 5 rewrite method invocations. The general scheme of these rules is the same as the previous one. Rule 5 just adds a variable assignment to rule 4. Rule 4 mainly passes three parameters: a function reference, arguments that is passed to this function, and the object to where the function belongs. This call uses these parameters to execute the original computation (i.e. proceed in AOP). Method Definition is rewritten as a closure by using wrap (rule 6). Instead of the original computation, the method

Statement

$$s = \textsf{call-s}(null,\ fun()\{s\},\ [],\ cn,\ mn); \quad (1)$$

$$\textsf{var}\ id{=}e; = \textsf{var}\ id;\ \textsf{call-s}(null,\ fun()\{id{=}e;\},\ [],\ cn,\ mn); \quad (2)$$

Object Creation

$$\textsf{var}\ id = new\ Cls(args) = \textsf{var}\ id;\ \textsf{call}(null,\ fun()\{id{=}new\ Cls(args)\},\ [],\ cn,\ mn); \quad (3)$$

Method Invocation

$$obj.m(args) = \textsf{call}(obj,\ obj.m,\ [args],\ cn,\ mn); \quad (4)$$

$$\textsf{var}\ id{=}obj.m(args) = \textsf{var}\ id;\ \textsf{call}(obj,\ obj.m,\ [args],\ cn,\ mn);\ \textsf{call}(null,\ fun()\{id{=}obj.id\},\ [],\ cn,\ mn); \quad (5)$$

Method Definition

$$fun(\overline{args})\ \{\overline{s}\} = fun(\overline{args})\ \{\ \textsf{wrap}(this,\ fun(\overline{args})\ \{\overline{s}\},\ \textsf{args},\ cn,\ mn)\}; \quad (6)$$

Fig. 2. Rewriting function.

returns a closure in which original statements of the method are moved to the body of a new function.

4.2. *Runtime weaving*

A function that wraps an original piece of code as a closure triggers the creation of a join point with its weaving. The SyncAS runtime provides the functions inserted by the code transformation. Listing 7 shows a simplified implementation of call.

The call method receives several objects as arguments: the target of the call(obj), the function being invoked (fun), the arguments to that function (args), the class name that contains the function (c_name) and name of the function (f_name). Every argument can be used for a programmer to precisely specify the join points that must be matched. In addition, the call method creates a call join point (C_JP) and invokes the weave method for weaving. The continuation management in Line 4 of Listing 7 is explained in Sec. 4.3.

```
1
2 function call(obj, fun, args, c_name, f_name):void {
3   var jp = new C_JP(obj, fun, args, c_name, f_name);
4   // continuation management
5   weave(jp);
6 }
```

Listing 7. A simplified `call` for weaving.

```
1  function weave(jp):* {
2    // extract aspect with _before_ advice kind
3    var asps = aspectList.filter(function(asp) {
4      return (asp.eval(jp, env, BEFORE));
5    });
6
7    // execute advice of filtered aspects
8    asps.map(function(asp) {
9      asp.adv(jp, env);
10   }
11
12   // execute the original computation
13   var result = jp.proceed();
14
15   // extract aspect with _after_ advice kind
16   asps = aspectList.filter(function(asp) {
17     return (asp.eval(jp, env, AFTER));
18   });
19
20   // execute advice of filtered aspects
21   asps.map(function(asp) {
22     asp.adv(jp, env);
23   }
24
25   // returning the result of the original computation
26   return result;
27 }
```

Listing 8. A simplified SyncAS for weaving.

Listing 8 is a simplified version the SyncAS weaving. The weave method receives the join point passed as argument. Lines 4–6 extract aspects that match the join point, whose advice kinds are *BEFORE*. This method invokes a pointcut that programmers specified. Next, the extracted aspects invoke their corresponding advices passing the join point and an environment of bindings as arguments. The original computation is run by invoking proceed then its return value (if one exists) is returned to the join point. Before this return, aspects, whose advice kinds are *AFTER*, are extracted and applied the same way as *BEFORE*.

4.3. *Runtime join point queue management*

Figure 3 shows two big processes: weaving and queuing a join point. The second process stores join points in multiple queues when the blocked(executionContext) flags is true. By default, this flag is false for every method execution, meaning that every statement is executed through the weaving process. When an aspect virtually blocks a method execution (Line 7 of Listing 6), this flag changes to true for this execution context, and the next join points of this context are stored in a queue. The first join point that is stored is named *blocking join point* (bjp), which is used to identify a queue (e.g. bjp_x identifies the queue jp_1, jp_2, jp_3). These stored join points represent continuations of the remaining executions of a method. As SyncAS does not really

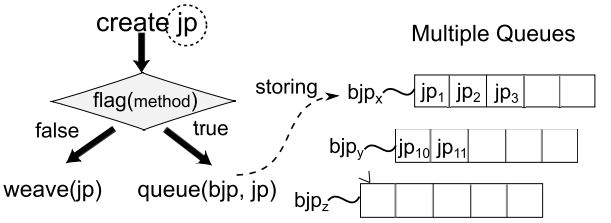

Fig. 3. SyncAS runtime.

block a program execution (just blocks a method execution), it is possible to keep many queues.

When a method execution must be restarted (Line 18 of Listing 6), the flag for this execution context changes to false, and the associated bjp to this execution is used to extract the join points of the queue (e.g. bjp$_x$ and jp$_1$, jp$_2$, jp$_3$). The extracted join points are processed by the SyncAS process.[b]

5. Evaluation

This section evaluates SyncAS from several aspects. Section 5.1 describes that SyncAS enables programmers to enhance a module based development in applications. Section 5.2 shows that SyncAS possess enough expressiveness compared to existing solutions being adopted by JavaScript communities. Section 5.3 represents the runtime overhead that SyncAS has to burden by conducting a micro-experiment.

5.1. *Modularity*

As we have described in Sec. 2.2, modular programming is natural for large-scaled applications. In this development, interfaces make loosely-coupled relation among modules, resulting that programmers are able to concentrate on their tasks. However, as described in Sec. 3.3, introducing non-blocking operation breaks this advantage. That is, a programmer has to modify its implementation when the another module implementation changes even if the module interface never changed. SyncAS enables encapsulation of pieces of code that handle control flow in one module (i.e. aspect), meaning that a programmer does not have to modify the implementation. Moreover, according to the discussion in Sec. 3.3, this aspect can be written by a non-programmer who uses an asynchronous method but a programmer who provides the method. This feature allows a programmer, who only uses an asynchronous method, to not have to worry to know whether or not methods are synchronous or asynchronous. In a large-scaled development, the each programmers' skills can be

[b] In SyncAS, a potential race condition could arise when multiple join point queues are processed in an arbitrary order. This race condition really comes from asynchronous requests of this style of programming. This issue is beyond the scope of this paper and Leger *et al.* provides a solution in [14].

different and asynchronous programming is more difficult than synchronous programming. Therefore, by using SyncAS, dividing tasks among programmers can be easier: expert programmers who know the asynchronous programming provide asynchronous methods by using non-blocking operations and create aspects to control the program execution, and non-expert programmers just use provided methods without considering whther or not they are synchronous or asynchronous.

5.2. *Flexibility to express existing solutions*

With the growth of Web technologies, JavaScript has been identified as the most dominant language on the Web [5]. The main focus of JavaScript was to provide the means to improve user interactions. For this reason, JavaScript provides support for user interactions employing an event-driven style, leading asynchronous issues.

One approach to deal with this problem adopted by JavaScript communities is the *Promise pattern* [7]: a proxy object that represents an unknown result that is yet to be computed. The common term used for JavaScript promises are the *thenable*, as a programmer uses a then method to attach a callback method, without blocking the program execution, To a promise when it is fulfilled. Future and Completer of the Dart language [4] adopt a similar approach to promise. Listing 9 shows the rewritten code of Listing 3 by using then. Line 1 of Listing 9 shows a function named send that binds url in its lexical scope, then this function creates a promise object. The function that is passed as an argument to the promise object, uses a non-blocking operation that invokes the resolve method (in Line 5) when the operation is completed. Using a promise object, a programmer can write a set of asynchronous methods with a fluent interface as shown in Line 13. In this case, checkAndConvertToImage is executed when resolve is invoked. In addition, then also returns a promise, implying that both promises can be chained together. The use of promise improves the understandability of a control flow through the use of fluent interface even if asynchronous methods are used.

In most libraries, promises are higher-level abstractions. Therefore in addition to chaining, they are composed easily. A programmer can write a number of high-level

```
1  function send(url:URL) {
2    return new Promise(function(resolve:Function) {
3      var dl = new Downloader();
4      dl.addEventListener(function(e) {
5        resolve(e.data);
6      });
7
8      dl.downloadasync(url);
9    });
10 }
11
12 var request = new Request();
13 request.send(url).then(checkAndConvertToImage).done();
```

Listing 9. The remote viewer version with then of promise.

```
var request = [req1.send(url1), req2.send(url2)];

function end(value) {
  console.log(value);
}

Promise.all(request).then(end);
```

Listing 10. An example of all of promise.

operations on promises by composing a group of independent asynchronous methods as shown in Listing 10. This listing creates two request objects (req1 and req2) and these objects download different data through their promises. The all method receives a list of promises then executes them in parallel. Finally, done is invoked when all promises in the list are completed. This abstraction provides a nicer interface with programmers compared to naive callback methods.

We now rewrite these listing with SyncAS. The then method represents a sequential execution in which every function passed to then waits until a function prior to it finishes. As we explained in Sec. 3.1, SyncAS enables a programmer to write asynchronous methods as a synchronous fashion.

Listing 11 represents a parallel execution with SyncAS, and Listing 12 shows the internal and hidden implementation necessary to execute the previous listing. Although lines 3–4 of Listing 11 looks a sequential execution, an aspect shown in Listing 12 enables lines 3-4 to be executed in parallel. This listing needs to wait for the execution of end until two asynchronous send methods finish. In order to count the number of these completions, the aspect uses a variable named counter that is stored in the environment. A pair of pointcuts and advices in line 2–8 of Listing 12 that matches when Line 3 of Listing 11 is executed initializes the counter. Another pair of pointcuts and advices in lines 12–19 of Listing 12 is used to virtually block the execution when this program reaches Line 4 of Listing 11. With this block, Line 5 of Listing 11 is never executed until the block is released. One of the last two pointcuts in lines 23 and 34 of Listing 12 matches when Line 3 of Listing 11 is completed and the other does when Line 4 of Listing 11 is finished.

When the last two pointcuts match join points, the same advice in Line 28 of Listing 12 is triggered. This advice increases the counter, and if the value equals 2, which represents the number of asynchronous methods, this advice invokes notify in order to release the block, and executes the end method.

```
1 var req1  = new Request1();
2 var req2  = new Request2();
3 req1.send(url1);
4 req2.send(url2);
5 end();
```

Listing 11. A parallel execution with SyncAS.

```
// initialize counter
var pcInit = function(jp,env) {
  return jp.isCall() && jp.className == "Request1"
                     && jp.methodName == "send";
}
var advInit = function(jp,env) {
  env.add("count",0);
}

// virtually block
var pcBlock = function(jp,env) {
  return jp.isCall() && jp.className == "Request2"
                     && jp.methodName == "send";
}
var advBlock = function(jp,env) {
  SyncAS.blockJP(jp);
  env.add("blockjp", jp);
}

// release if count == 2 (the number of executions)
var pcRestart1 = function(jp,env) {
  return jp.isExec() && jp.className == "Request1"
                     && jp.methodName == "complete";
}
var advRestart = function(jp,env) {
  env = env.add("count" , ++(env.get("count")));
  if (env.get("count") == 2)
    SyncAs.restartJP(env.get("blockjp"));
}
var pcRestart2 = function(jp,env) {
  return jp.isExec() && jp.className == "Request2"
                     && jp.methodName == "complete";
}

var virtualBlock = new Aspect([pcInit,advInit,BEFORE],
                              [pcBlock,advBlock,AFTER],
                              [pcRestart1,advRestart,AFTER],
                              [pcRestart2,advRestart,AFTER]);

SyncAS.deploy(virtualBlock);
```

Listing 12. An aspect for a parallel execution.

As we have described so far, the nature of SyncAS allows a programmer to virtually block and restart an execution of a program at an arbitrary point. Therefore, it is possible to control the execution flow as *promise* through the definition of an appropriate aspect. The previous point implies SyncAS is expressive enough to be (at least) applicable to the Web development.

5.3. *Runtime performance*

Although the purpose of SyncAS is to provide programmers with abilities to control program executions by using aspects, one of the drawbacks can be its runtime performance because of the runtime weaving. Using an Intel Core i7 (2.7GHz), 16GB of

RAM and Apache Flex 4.9.1 running on MacOSX (10.8.5), we measured the total execution time (ms). Concretely, we conducted two experiments:

(1) We created a number of join points (100; 1000; 10,000; 100,000), which are "seen" by two aspects: one aspect that matches all join points and another that does not match any join point. In addition, we conducted the same range of method invocations without the creation of join points to measure the effect on their creations. Note that the creation of a join point is to generate an additional method invocation because SyncAS wraps each statement with a closure (Sec. 4.1).
(2) We deployed different numbers of aspects (1, 10, 100) with a fix set of 1,000 join points.

Figures 4 and 5 show results of previous experiments, where y-axis uses a logarithmic scale to show the effect multiplicative factors of runtime performance. Figure 4 shows that SyncAS works with scalability. In addition the effect of an advice execution is limited (about 20–25% slower), however the join point creation and the aspect deployment impacts the performance (making it about 3 times slower). This is because one join point creation triggers three implicit additional method invocations: making a closure, extracting an aspect (e.g. aspectList in Listing 8), and evaluating a pointcut. As shown in Fig 5, the increase in aspect instances does not impact performance compared to the join point creation. Because this increase just adds one additional implicit method invocation (i.e. a pointcut execution).

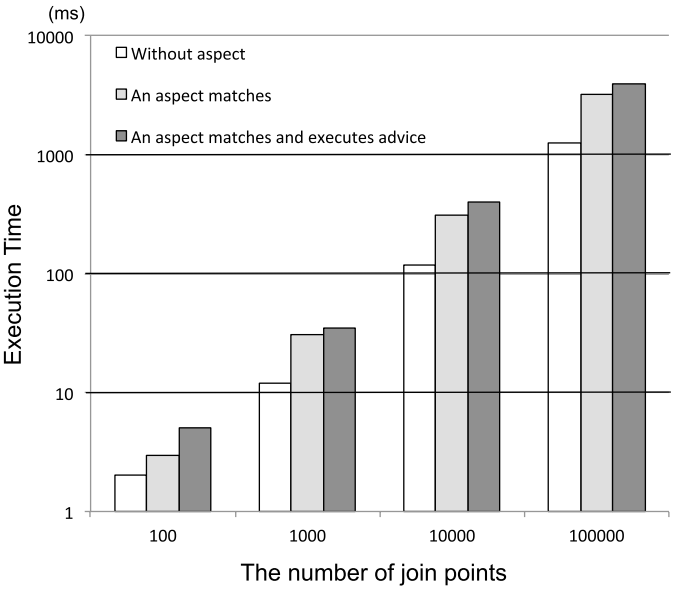

Fig. 4. Execution time based on the number of join points.

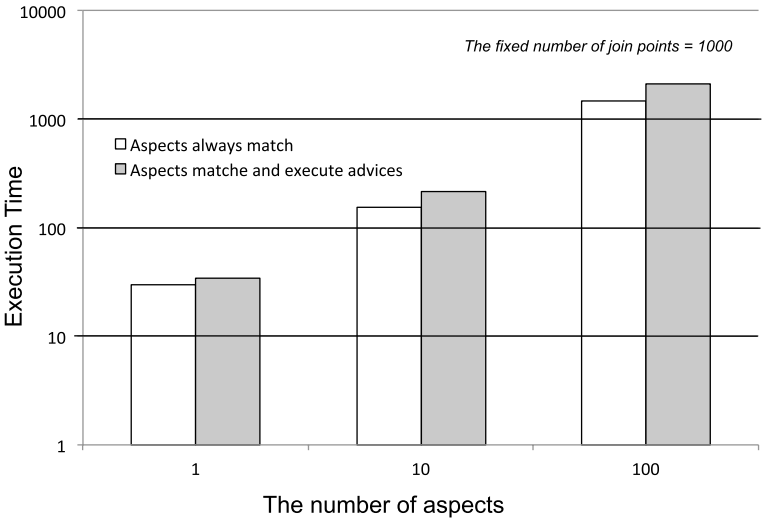

Fig. 5. Execution time based on the number of aspects.

The execution of join points in these experiments is synchronous, therefore raw overhead in these CPU-intensive scenarios do not directly impact performance of practical applications in which asynchronous methods are adopted to improve the user interaction. To verify this scenario, we conducted a small experiment in which a 11kbyte sized image is downloaded using a single computer, meaning that a client can connect to a server with high speed network. This experiment took about 300 (ms). This amount of time corresponds to from 1,000 to 10,000 join points in Fig 4. This means that, to be simplified, this amount of time corresponds to from 1,000 to 10,000 statement executions because every statement is wrapped by a closure in SyncAS. The amount of 1,000 statements in a single event handler is not adequate in a practical application, meaning that the effect of creating join points is limited in practical applications.

Apart from practical scenarios for which SyncAS can be applied, a SyncAS optimization is able to improve its runtime performance. For example, in these experiments, we assume that all executions are wrapped by closures, negatively impacting performance. Implementing a code instrumentation tool that gives partial transformations can significantly improve the SyncAS runtime performance. This is because this code instrumentation will transform only parts of a program that use asynchronous methods, resulting in the reduction of implicit method invocations in the SyncAS runtime.

6. Related Work

The purpose behind applying asynchronous programming is the concurrency provided. The concurrency support varies from runtime systems to libraries. For

example design patterns such as the *Promise pattern* described in Sec. 5.2 is one of the solutions that improves the way of writing asynchronous programming. We now review different levels of supports: system, language, and application of novel paradigms.

System level. As well as event-driven style, thread is one of the methods to enforce asynchronous programming. It is usually supported by runtime systems (e.g. JavaVM). In [13], the authors note that the thread-based and event-based models are dual: a program written in one style can be transformed to another style. Although threads provide the means to write a set of asynchronous executions as synchronous fashion, avoiding *callback hell*, it leads to other problems. Ousterhout in [18] states that threads are more difficult to program than events because the programmers must reason about a shared state. SyncAS can be applied to the event-based style with single-threaded applications, meaning that the programmers do not need to consider the drawbacks of threads while avoiding issues such as *callback hell*.

Language level support. C# 5.0 [2, 3] provides *async/await* constructs that supply a mechanism to write programs in a synchronous fashion. An asynchronous method attached *await* keeps the following executions as continuations that are restarted when the asynchronous method is completed (details explained in "explicit specification" of Sec. 3.2). A method definition with *async* modifier lets the compiler know what methods include asynchronous executions inside. However, programmers who use asynchronous methods need to know which are asynchronous methods when they write a program, meaning that crosscutting concerns can be scattered and tangled in main concerns. Instead, with SyncAS, non-experienced programmers do not need to consider the differences between asynchronous and synchronous methods. Then a piece of code that controls program executions is written in a module (i.e. aspect).

Aspect-Oriented Programming. In [6], the authors use AOP design patterns to the implementation of concurrent programming. The paper proposes an aspect to encapsulate pieces of code that prevent the reusability in thread-based concurrent programming. This aspect contains two pairs of pointcuts and advices. The first one intercepts any async operation and returns a promise value. The second one intercepts the access of a promise value and blocks the program execution until the promise value is resolved. This proposal is similar to SyncAS (Listing 6). However, the authors' approach really blocks the control of the thread this approach is applied until the promise value is resolved, meaning that the control may not be returned immediately because of the real block. As a consequence, if the base language does not support the multithread, all program executions are blocked. Instead, SyncAS does not perform a real block of the program execution but it only blocks the execution of one method. In addition, SyncAS does not only provide a design pattern, but a working mechanism, which includes the management of join point queues, to control partial blocking of an application. Therefore, SynAS can be applied to highly

responsive applications such as Web applications even if the base language only supports single threads.

7. Conclusion

Asynchronous programming has been widely adopted for various reasons. Callback is a typical solution that enables asynchronous programming. However, this solution has its drawbacks such as *callback spaghetti* – the modern goto statement in asynchronous programming, and *callbakc hell* – deeply nested anonymous functions. In addition, introducing asynchronous programming into module based programming requires dividing a method into call-site and its continuations, making complex control flows. In order to solve these drawbacks, this paper proposes a modular library called SyncAS that enables programmers to virtually block the execution of a program at arbitrary points and restart it by using aspect-oriented techniques. In this paper, we describe the basic idea and provide a SyncAS prototype implementation in ActionScript. This implementation was used to explain the advantages by comparing existing solutions. With SyncAS, a programmer can write asynchronous executions as synchronous style, meaning that the programmer does not have to worry about an invocation being asynchronous or synchronous.

Finally, we point out future issues to be solved such as applying SyncAS to practical applications. As we described in Sec. 4, we first implemented the Syncas runtime and designed a code transformation rules, though the transformation was manually done in this paper. Therefore we need to implement a code transformation tool that enables the automatic transformation with the optimization that will help the runtime performance. Second, we have verified the behavior and runtime performance of SyncAS with toy examples, however, it is not enough to show the use of SyncAS for practical applications. Therefore, we will apply SyncAS to existing practical applications that use complex asynchronous executions.

Acknowledgments

This work was partially supported by JSPS KAKENHI Grant No. 26330089.

References

1. T. Azegami, H. Fukuda and P. Leger, Towards a virtual block approach to tame asynchronous programming, in *Proceedings of Modularization for Practical Software Engineering*, 2014.
2. N. Benton, L. Cardelli and C. Fournet, Modern concurrency abstractions for C#, *ACM Trans. Program. Lang. Syst.* **26**(5) (2004) 769–804.
3. G. Bierman, C. Russo, G. Mainland, E. Meijer and M. Torgersen, Pause 'n' play: Formalizing asynchronous C#, in *Proc. 26th European Conference on Object-Oriented Programming*, 2012, pp. 233–257.
4. C. Buckett, *Dart in Action* (Manning, 2013).

5. D. Crockford, The world's most misunderstood programming language has become the world's most popular programming language, http://javascript.crockford.com/popular.html.
6. C. A. Cunha, J. A. L. Sobral and M. P. Monteiro, Reusable aspect-oriented implementations of concurrency patterns and mechanisms, in *Proc. 5th International Conference on Aspect-Oriented Software Development*, 2006, pp. 134–145.
7. D. Friedman and D. Wise, *The Impact of Applicative Programming on Multiprocessing*, Technical report, Indiana University, Computer Science Department, 1976.
8. H. Fukuda and P. Leger, A library to modularly control asynchronous executions, in *Proc. 29th Annual ACM Symposium on Applied Computing*, 2015.
9. D. Holroyd, MetaAS: Java library for manipulating actionscript source code, http://www.badgers-in-foil.co.uk/projects/metaas/.
10. K. Kambona, E. G. Boix and W. De Meuter, An evaluation of reactive programming and promises for structuring collaborative web applications, in *Proc. 7th Workshop on Dynamic Languages and Applications*, 2013, pp. 3:1–3:9.
11. G. Kiczales, E. Hilsdale, J. Hugunin, M. Kersten, J. Palm and W. Griswold, An overview of AspectJ, in *Proc. 15th European Conference on Object-Oriented Programming*, LNCS 2001, Vol. 2072, pp. 327–353.
12. G. Kiczales, J. Irwin, J. Lamping, J. Loingtier, C. Lopes, C. Maeda and A. Mendhekar, Aspect Oriented Programming, in *Special Issues in Object-Oriented Programming*, 1996.
13. H. C. Lauer and R. M. Needham, On the duality of operating system structures, *SIGOPS Oper. Syst. Rev.* **13**(2) (1979) 3–19.
14. P. Leger, É. Tanter and R. Douence, Modular and flexible causality control on the web, *Science of Computer Programming* **78**(9) (2013) 1538–1558.
15. H. Masuhara, G. Kiczales and C. Dutchyn, A compilation and optimization model for aspect-oriented programs, in G. Hedin (ed.), *Proc. Compiler Construction*, 2003, pp. 46–60.
16. T. Mikkonen and A. Taivalsaari, Web applications — spaghetti code for the 21st century, in *Proc. Sixth International Conference on Software Engineering Research, Management and Applications*, 2008, pp. 319–328.
17. M. Ogden. Callback hell. http://callbackhell.com/.
18. J. Ousterhout, Why threads are a bad idea (for most purposes), in *USENIX Winter Technical Conference*, June 1996.
19. D. L. Parnas, On the criteria to be used in decomposing systems into modules, *Commun. ACM* **15**(12) (1972) 1053–1058.
20. R. Toledo, P. Leger and E. Tanter, Aspectscript: Expressive aspects for the web, in *Proc. 9th International Conference on Aspect-Oriented Software Development*, 2010, pp. 13–24.